

SmartBOQ Pro

AI-Powered Construction Estimation & Visual Inspection System

Complete Technical Documentation, Interview Guide & Project Report

Repository	github.com/rockyrahul4143-source/AI-Based-Crack-Detection-in-Concrete
Technology Stack	Python 3.12 FastAPI React 18 TensorFlow 2.21 SQLite
AI Models	ResNet50 + VGG16 + InceptionV3 Ensemble MobileNetV2
Model Accuracy	99.58% Validation Accuracy on Crack Detection
Dataset	40,000 Images (20K Positive + 20K Negative)
Currency	INR (Indian Rupee) - India Market
Generated	16 July 2026, 12:47 AM

1. Project Overview

1.1 Project Title

SmartBOQ Pro — AI-Powered Construction Estimation & Visual Inspection System

1.2 Project Objective

To automate construction project estimation, bill of quantities generation, and structural defect detection using deep learning computer vision models — replacing manual site inspection and spreadsheet-based estimation with an intelligent web application built for the Indian market.

1.3 Problem Statement

- Manual estimation is slow, error-prone, and inconsistent across projects
- Site inspections for cracks/defects require expensive engineers on-site
- BOQ generation takes days and is prone to human calculation errors
- 30-40% of construction projects in India face cost overruns due to poor estimation
- Concrete cracks cause structural failures worth crores annually in India
- No affordable AI-powered tool existed for small/medium Indian contractors

1.4 Why This Project Was Developed

The construction industry is one of the largest in India yet remains one of the least digitized. Academic research had produced excellent crack detection models in Jupyter notebooks, but these were never deployed as production-ready tools. This project bridges that gap by wrapping trained deep learning models into a full-stack web application with a professional UI, REST API, database, and export capabilities.

1.5 Real-World Applications

Application	Sector	Example Use
Structural health monitoring	Civil Engineering	Monitor columns, beams, slabs for cracks
Site quality control	Construction	Daily photo upload to track defects
Insurance claim verification	Insurance	Assess damage after earthquake/flood
Government road inspection	PWD / NHAI	Pothole detection on national highways
Smart city monitoring	Urban Planning	Automated bridge/flyover inspection
Property pre-purchase	Real Estate	Inspect before buying a flat/house
Cost estimation	Contracting	Bid preparation for tenders

2. Technology Stack

2.1 Programming Languages

Language	Where Used	Why Chosen
Python 3.12	Backend, ML, Training scripts	Best language for AI/ML, FastAPI support, rich ecosystem
TypeScript	Frontend (React)	Type safety catches bugs at compile time, better IDE support
SQL	Database queries (via ORM)	Data storage and retrieval
CSS (Tailwind)	Frontend styling	Utility-first, no separate CSS files needed
Batch Script	start.bat, stop.bat	Windows automation for one-click launch

2.2 Backend Libraries

Library	Version	Purpose
FastAPI	0.111.0	REST API framework with auto Swagger docs
SQLAlchemy	2.0.30	ORM — maps Python classes to DB tables
Pydantic	2.7.1	Automatic request/response validation
python-jose	3.3.0	JWT token creation and verification
passlib + bcrypt	1.7.4	Password hashing (one-way, secure)
TensorFlow	2.21.0	Deep learning model inference
Pillow	10.3.0	Image loading, resizing, conversion
NumPy	1.26.4	Array operations for image preprocessing
ReportLab	4.2.0	PDF report generation
OpenPyXL	3.1.4	Excel BOQ export
ezdxf	1.3.3	AutoCAD DXF file parsing
uvicorn	0.30.1	ASGI web server to run FastAPI
alembic	1.13.1	Database schema migration tool

3. System Architecture

The system follows a clean 3-tier architecture: Presentation (React frontend), Application (FastAPI backend with services layer), and Data (SQLite + file system).

Layer	Technology	Port	Responsibility
Frontend	React 18 + TypeScript + Vite	5173	User interface, forms, charts, file upload
Backend API	FastAPI + Python 3.12	8000	11 REST API routers, JWT auth, business logic
Services	TensorFlow, ReportLab, ezdxflib	-	AI inference, PDF/Excel generation, DXF parsing
Database	SQLite + SQLAlchemy ORM	-	Users, projects, estimates, BOQ, inspections
ML Models	Keras .h5 files	-	Trained weights for 4 inspection types

API Flow: User uploads image → React sends multipart form to FastAPI → ai_inspection.py preprocesses image → Model runs inference → Result saved to DB → JSON response returned to frontend → UI displays severity + repair cost estimate.

4. Folder Structure & File Purposes

File / Folder	Purpose
start.bat	One-click launcher: kills old servers, runs setup_local.py, starts backend + frontend
stop.bat	Kills all SmartBOQ server processes
docker-compose.yml	Production deployment with Docker containers
1.ipynb	Jupyter notebook: ResNet50+VGG16+InceptionV3 ensemble training (99.58% accuracy)
2.ipynb	Jupyter notebook: MobileNetV2 road damage/pothole detection training
backend/train_fast.py	Automated training script: trains all 4 models from Datasets/ folder
backend/setup_local.py	DB initialization: creates tables + seeds 46 materials + admin user (first run only)
backend/start_local_py312.py	Startup: loads TF models into memory, starts uvicorn server on port 8000
backend/requirements.txt	All Python dependencies with pinned versions
backend/ml_models/*.h5	Trained model weight files (not in git - too large, download separately)
backend/app/main.py	FastAPI app factory: creates app, adds CORS middleware, mounts all 11 routers
backend/app/core/config.py	Pydantic Settings: reads DATABASE_URL, SECRET_KEY, etc. from .env
backend/app/core/security.py	JWT functions: create_access_token(), hash_password(), verify_password()
backend/app/core/dependencies.py	FastAPI dependencies: get_current_active_user(), RBAC checks
backend/app/db/base.py	SQLAlchemy: creates engine, SessionLocal, get_db() dependency
backend/app/models/inspection.py	Inspection ORM model + InspectionType + SeverityLevel enums
backend/app/services/ai_inspection.py	Core AI service: loads models, preprocesses images, runs inference, maps severity
backend/app/services/estimation_engine.py	Quantity calculation formulas for 10+ work types
backend/app/services/report_generator.py	Generates PDF reports using ReportLab with tables and formatting
backend/app/api/v1/inspection.py	AI inspection REST endpoints: /concrete-crack, /road-damage, /history, /stats
frontend/src/lib/api.ts	Axios instance with base URL + JWT auth interceptor (auto-adds Bearer token)
frontend/src/context/AuthContext.tsx	Global auth state: login(), logout(), token refresh, user object
frontend/src/pages/InspectionPage.tsx	Image upload UI + AI result display with severity badges
frontend/src/pages/EstimationPage.tsx	Work type selection + dimension inputs + cost calculation
frontend/src/pages/DashboardPage.tsx	Statistics cards + Recharts bar/line/pie charts
docs/API_REFERENCE.md	Complete API documentation with all endpoints, request/response schemas
docs/ARCHITECTURE.md	System design, database schema, deployment architecture

5. Dataset & Machine Learning

5.1 Dataset Details

Property	Details
Total Images	40,000 (balanced dataset)
Positive (Crack)	20,000 images of concrete with cracks
Negative (No Crack)	20,000 images of normal concrete
Format	JPG images
Location	Datasets/Positive/ and Datasets/Negative/
Train Split	80% (32,000 images)
Validation Split	20% (8,000 images)
Preprocessing	Resize to 120x120 or 160x160, normalize to [0,1]

5.2 Preprocessing Steps

- Step 1: Load image using `PIL.Image.open().convert("RGB")`
- Step 2: Resize to target size (120x120 for ResNet/VGG/Inception, 160x160 for MobileNetV2)
- Step 3: Convert to NumPy array with `dtype=float32`
- Step 4: Normalize — divide by 255.0 to get values in range [0, 1]
- Step 5: `train_test_split` with `test_size=0.2`, `stratify=y` (equal class distribution)

Normalization is critical: neural networks converge faster and more stably with normalized inputs.

5.3 Model Architecture

All models use Transfer Learning from ImageNet pre-trained weights. The base model is frozen initially (feature extraction), then the last 30 layers are unfrozen for fine-tuning.

Model	Input Size	Params	Val Accuracy	File Size	Best For
ResNet50	120x120	25M	99.58%	93.6 MB	General crack detection
VGG16	120x120	138M	99.5%	57 MB	Fine texture detection
InceptionV3	150x150	23M	99.4%	87.1 MB	Multi-scale feature detection
MobileNetV2	160x160	3.4M	99.58%	20.8 MB	Road damage (lightweight)

5.4 Custom Classification Head (added to each base model)

- `GlobalAveragePooling2D` — reduces feature maps to 1D vector, fewer parameters than `Flatten`
- `BatchNormalization` — normalizes activations, speeds up training, improves stability
- `Dense(128, relu)` — learns crack-specific patterns from extracted features
- `Dropout(0.4)` — randomly drops 40% of neurons during training to prevent overfitting
- `Dense(1, sigmoid)` — outputs probability between 0 and 1 (binary classification)

5.5 Training Configuration

Parameter	Value	Reason
Optimizer	Adam (lr=1e-3, then 1e-5)	Adaptive learning rate, fastest convergence
Loss Function	Binary Cross-Entropy	Standard for binary classification
Metrics	Accuracy	Easy to interpret for binary classification
Batch Size	64	Fits in RAM, good gradient estimates

Parameter	Value	Reason
Max Epochs	10 + 5 fine-tune	With EarlyStopping, stops when accuracy plateaus
EarlyStopping	patience=3, monitor=val_accuracy	Stops if no improvement for 3 epochs
Phase 1	Base frozen, train head only	Learn task-specific features quickly
Phase 2	Last 30 layers unfrozen	Fine-tune deep features for crack detection

6. Features Implemented

#	Feature	Description
1	JWT Authentication	Login, register, refresh token, forgot/reset password
2	Role-Based Access	Admin, Estimation Engineer, Quantity Surveyor, Project Manager
3	Project Management	Full CRUD: create, edit, delete projects with status tracking
4	Building & Rooms	Multi-floor buildings with rooms and auto area/volume calculation
5	Estimation Engine	10+ work types, auto quantity calculation, material+labour costs
6	BOQ Generator	Line items, profit%, contingency%, grand total, INR currency
7	AI Inspection - Concrete	ResNet50+VGG16+InceptionV3 ensemble, 99.58% accuracy
8	AI Inspection - Road	MobileNetV2 pothole/damage detection
9	Severity Scoring	Low/Moderate/High/Critical with repair urgency and cost range (INR)
10	Materials Database	46 pre-seeded materials with 2024 India market rates
11	PDF Export	Professional BOQ/estimate/inspection reports via ReportLab
12	Excel Export	BOQ spreadsheet export via OpenPyXL/XlsxWriter
13	DXF Parser	Upload AutoCAD .dxf files, auto-extract room dimensions
14	Analytics Dashboard	Charts: cost trends, project status, material analysis
15	Inspection History	Filterable history with severity stats per project
16	Audit Logging	Every user action logged to AuditLog table
17	Dark/Light Mode	Theme toggle stored in context
18	API Documentation	Auto-generated Swagger UI at /docs

7. Security & Performance

7.1 Security Measures

- Passwords — bcrypt hashed with salt. Never stored in plain text. Cannot be reversed.
- JWT Tokens — 30-minute access token + 7-day refresh token. Stateless authentication.
- SQL Injection — Prevented entirely by SQLAlchemy ORM parameterized queries.
- CORS — Only http://localhost:5173 allowed in development. Configurable for production.
- File Upload — Extension whitelist: .jpg, .jpeg, .png, .bmp, .webp only.
- File Size — Maximum 50MB per upload enforced at API level.
- RBAC — Admin-only endpoints protected by FastAPI dependency injection.
- Input Validation — Pydantic validates every request body automatically.

7.2 Performance Optimizations

- Model Loading — All 4 TF models loaded once at startup, cached in `_models` dict in memory.
- Image Processing — Done entirely in-memory using BytesIO. No temporary files written.
- Process Time Header — X-Process-Time header added to every response for monitoring.
- EarlyStopping — Training stops automatically when validation accuracy plateaus.
- Transfer Learning — Uses pre-trained ImageNet weights, reducing training time by 90%.

8. Interview Preparation — 50 Technical Q&A;

Q1. What is SmartBOQ Pro in one sentence?

An AI-powered full-stack web application that automates construction cost estimation, BOQ generation, and structural defect detection using deep learning models achieving 99.58% validation accuracy.

Q2. What is Transfer Learning and why did you use it?

Transfer learning reuses a model trained on one large dataset (ImageNet - 14M images) for a different but related task (crack detection). I froze the base model weights and added a custom classification head. This achieved 99.58% accuracy with only 40,000 images. Training from scratch would need millions.

Q3. What is the difference between ResNet50, VGG16, and InceptionV3?

VGG16: Simple 16-layer sequential network, 138M parameters, easy to understand, large model size. ResNet50: 50 layers with skip/residual connections that solve the vanishing gradient problem, 25M params. InceptionV3: Uses parallel convolutions (1x1, 3x3, 5x5) to capture multi-scale features efficiently, 23M params.

Q4. What is an ensemble model and why did you use one?

Running multiple models on the same input and averaging their predictions. My ensemble averages sigmoid outputs of ResNet50+VGG16+InceptionV3. If one model makes an error, the others correct it. This reduces variance and improves robustness. Ensemble accuracy > any single model accuracy.

Q5. What is a CNN and how does it detect cracks?

Convolutional Neural Network applies learnable filters that slide over images. Early layers detect edges and colors. Middle layers detect shapes and textures. Deep layers detect high-level patterns like cracks. Pooling reduces spatial dimensions. Final dense layers classify as crack/no-crack.

Q6. What is sigmoid activation?

$\text{sigmoid}(x) = 1/(1+e^{-x})$. Maps any real value to (0,1). Output = probability of positive class (crack). If output > 0.5 = crack detected. Perfect for binary classification. For multi-class, use softmax instead.

Q7. What is binary cross-entropy loss?

Loss = $-[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$ where y =true label, p =predicted probability. Penalizes confident wrong predictions very heavily (log of near-zero = large negative number). Standard loss function for binary classification problems.

Q8. What is Dropout and why use 0.4?

Dropout randomly sets 40% of neurons to zero during training. Forces network to learn redundant representations. Prevents overfitting (memorizing training data). At inference time, all neurons active. 0.4 is a good default for middle dense layers — high enough to prevent overfitting, low enough to learn.

Q9. What is EarlyStopping?

Keras callback that monitors validation accuracy. Stops training if it does not improve for patience=3 epochs. Restores best weights automatically. Prevents overfitting and saves training time.

Q10. What is GlobalAveragePooling2D vs Flatten?

Flatten: converts all spatial positions to 1D vector. For 7x7x2048 feature map = 100,352 parameters. GlobalAveragePooling2D: takes average of each 7x7 map = 2048 parameters. Much fewer params, less

overfitting, works as structural regularizer.

Q11. What is BatchNormalization?

Normalizes layer inputs to mean=0, variance=1 using learned scale and shift parameters. Speeds up training, allows higher learning rates, reduces sensitivity to weight initialization, acts as slight regularizer.

Q12. What is FastAPI and why use it instead of Flask?

FastAPI advantages over Flask: Auto-generates Swagger/OpenAPI docs at /docs. Native async support for concurrent requests. Automatic validation via Pydantic. 3x faster than Flask in benchmarks. Built-in type hints throughout.

Q13. How does JWT authentication work in your project?

Login: verify email+password -> create JWT with user_id and expiry signed with SECRET_KEY. Client stores token, sends in every request: Authorization: Bearer . Server verifies signature and extracts user_id. No database lookup needed for each request. Access token: 30 min. Refresh token: 7 days.

Q14. What is bcrypt and why not store plain text passwords?

bcrypt applies one-way hash with random salt and cost factor. Cannot be reversed. Two identical passwords produce different hashes due to salt. If database is stolen, passwords cannot be recovered. Work factor can be increased as hardware gets faster.

Q15. What is SQLAlchemy ORM?

Maps Python classes to database tables. Prevents SQL injection by using parameterized queries. `db.query(User).filter(User.email == email).first()` generates safe SQL automatically. Database-agnostic: switch from SQLite to PostgreSQL by changing just the connection URL.

Q16. What is CORS and why is it needed?

Cross-Origin Resource Sharing. Browser blocks JS from calling APIs on different origins (domain+port). Frontend on port 5173 calls backend on port 8000 = different origin. Backend must send Access-Control-Allow-Origin: http://localhost:5173 header to permit this.

Q17. What is Pydantic validation?

Define Python class with type annotations. FastAPI automatically validates every request body. Invalid data returns HTTP 422 automatically. UserCreate schema validates email format, password length/complexity without writing manual validation code.

Q18. How does the severity scoring work?

Confidence threshold mapping: >0.90 = CRITICAL (immediate repair). >0.75 = HIGH (7 days). >0.55 = MODERATE (30 days). >0.50 = LOW (monitor). Each severity maps to INR repair cost range: CRITICAL = Rs.50,000-2,00,000. LOW = Rs.1,000-8,000.

Q19. What is the difference between .h5 and .pt model files?

.h5 is Keras/TensorFlow HDF5 format. Saves entire model: architecture + weights + optimizer state. Load with `tf.keras.models.load_model()`. .pt is PyTorch format. Saves state dict (weights only). Load with `torch.load()` + `model.load_state_dict()`. This project uses .h5 (TensorFlow/Keras).

Q20. Why did you use SQLite instead of PostgreSQL?

SQLite is a file-based database, zero configuration, perfect for local development and small teams. For production, simply change DATABASE_URL in .env to PostgreSQL connection string. SQLAlchemy ORM handles the rest automatically.

Q21. What is Vite and why use it over Create React App?

Vite uses native ES modules and esbuild (Go-based bundler). Dev server starts in <1 second vs 30+ seconds for CRA. Hot Module Replacement is near-instant. Production build is 10x faster. No webpack configuration needed.

Q22. What is Tailwind CSS?

Utility-first CSS framework. Instead of writing CSS files, apply pre-built utility classes directly in HTML/JSEX: `className="bg-blue-500 text-white p-4 rounded-lg"`. Purges unused CSS in production resulting in very small bundle sizes. Consistent design system.

Q23. How does the DXF parser work?

ezdxf library reads AutoCAD .dxf binary/ASCII format. Extracts LINE and LWPOLYLINE entities. Groups horizontal and vertical lines to detect room boundaries. Calculates room dimensions from line endpoints. Returns room names, dimensions, and areas to the estimation engine.

Q24. How does the estimation engine calculate quantities?

For masonry: volume = length x height x thickness x (1 - door/window openings). Convert to bricks count using standard brick size (230x115x75 mm) + mortar factor. Multiply by material rate from database. Add 10% wastage. Add labour cost from rate schedules.

Q25. What is ReportLab and how do you generate PDFs?

ReportLab is a pure-Python PDF generation library. Uses Flowables (Paragraph, Table, Spacer, Image) assembled into a document. SimpleDocTemplate handles page breaks and margins automatically. TableStyle applies colors, borders, padding to tables.

Q26. What is Role-Based Access Control in your project?

Four roles: Admin, Estimation Engineer, Quantity Surveyor, Project Manager. FastAPI dependency `get_current_active_user()` validates JWT and returns user object. Admin-only endpoints use additional `require_admin()` dependency. Non-admins get HTTP 403 Forbidden if they access admin routes.

Q27. What metrics did you use to evaluate the models?

Primary: Validation Accuracy (99.58% achieved). Also monitored: Validation Loss (0.076 final). Training accuracy to check for overfitting (train acc $\approx$ val acc means no overfitting). EarlyStopping monitored `val_accuracy` with `restore_best_weights=True`.

Q28. What is overfitting and how did you prevent it?

Overfitting: model memorizes training data, fails on new images. Prevention techniques used: Dropout(0.4), EarlyStopping, Transfer Learning (pre-trained features), Validation split (20%), BatchNormalization, GlobalAveragePooling2D instead of Flatten.

Q29. What is fine-tuning and how did you do it?

After training the classification head with frozen base, unfreeze the last 30 layers of the base model. Use very small learning rate ($1e-5$) to avoid destroying pre-trained features. Train for 5 more epochs. This adapts deep features specifically to crack patterns.

Q30. How does the React AuthContext work?

React Context provides global auth state. `Login()` calls `/api/v1/auth/login`, stores JWT in `localStorage`. Axios interceptor automatically adds Bearer token to every request. On 401 response, interceptor calls `/auth/refresh` to get new access token automatically.

9. Viva Questions & Answers (25)

Q: What is the output of the sigmoid function?

A: A value between 0 and 1 representing probability.

Q: Why 40,000 images and not fewer?

A: Deep learning needs large data. Fewer images = overfitting. 40K gives enough variation in lighting, angles, crack types.

Q: What happens if a model file is missing?

A: The API returns a `model_weights_not_found` response with setup instructions. The app does not crash.

Q: What is the purpose of the BatchNormalization layer?

A: Normalizes activations to mean=0, variance=1. Stabilizes and speeds up training.

Q: Why use Adam optimizer?

A: Adaptive learning rate per parameter. Faster convergence than SGD. Handles sparse gradients well.

Q: What is the role of the Dropout layer?

A: Randomly disables 40% of neurons during training to prevent memorization of training data (overfitting).

Q: What is ImageNet?

A: Dataset with 14 million labeled images across 1000 classes used to pre-train ResNet, VGG, Inception models.

Q: Why normalize pixel values to [0,1]?

A: Neural networks are sensitive to input scale. Large values (0-255) cause slow/unstable training. Normalized inputs converge faster.

Q: What is a confusion matrix?

A: 2x2 table: True Positive, False Positive, True Negative, False Negative. Used to calculate precision, recall, F1-score.

Q: What is precision vs recall?

A: Precision = $TP / (TP + FP)$ — of predicted cracks, how many were real? Recall = $TP / (TP + FN)$ — of real cracks, how many were found?

Q: What is F1 score?

A: Harmonic mean of precision and recall: $2 * (P * R) / (P + R)$. Useful when classes are imbalanced.

Q: What is the vanishing gradient problem?

A: In deep networks, gradients become very small during backpropagation, preventing early layers from learning. ResNet solves this with skip connections.

Q: What are skip connections in ResNet?

A: Shortcut connections that add the input of a block directly to its output. Gradient flows directly, solving vanishing gradient problem.

Q: What is backpropagation?

A: Algorithm to train neural networks. Computes gradient of loss with respect to each weight using chain rule. Updates weights to minimize loss.

Q: What is the difference between train and validation accuracy?

A: Train: accuracy on training data. Validation: accuracy on unseen data. If train >> val, the model is overfitting.

Q: What is an epoch?

A: One complete pass through the entire training dataset. Training typically runs for multiple epochs until loss stops improving.

Q: What is batch size?

A: Number of samples processed before weights are updated. Larger = faster but more memory. Smaller = noisier but better generalization.

Q: Why use binary cross-entropy for 2-class classification?

A: It directly measures the difference between predicted probability and true label (0 or 1). Penalizes confident wrong predictions heavily.

Q: What is the difference between supervised and unsupervised learning?

A: Supervised: labeled data (crack/no crack). Unsupervised: no labels, finds patterns (clustering). This project is supervised.

Q: What is the purpose of the CRUD layer?

A: Separates database operations from API routes. Makes code reusable, testable, and maintainable.

Q: How does Alembic work?

A: Database migration tool. Creates versioned SQL scripts to evolve schema without losing data. Like git for database structure.

Q: What is ASGI vs WSGI?

A: WSGI: synchronous Python web standard (Flask, Django). ASGI: asynchronous (FastAPI, Django Channels). ASGI handles concurrent connections better.

Q: What is the purpose of the .env file?

A: Stores configuration secrets (SECRET_KEY, DATABASE_URL) outside code. Never committed to git. Different values for dev/prod.

Q: What is Docker and how is it used here?

A: Containerization platform. docker-compose.yml defines backend + frontend + nginx containers. Ensures consistent environment on any machine.

Q: How would you scale this for 1 million users?

A: Replace SQLite with PostgreSQL. Add Redis for caching. Deploy on Kubernetes. Use GPU instances for faster model inference. Add load balancer.

10. How to Explain in Interviews

30-Second Introduction

"I built SmartBOQ Pro, an AI-powered construction management system. It uses deep learning — specifically ResNet50, VGG16, and InceptionV3 ensemble — to detect cracks in concrete structures with 99.58% accuracy. The system also automates construction cost estimation and BOQ generation. Built with FastAPI backend, React frontend, and TensorFlow."

1-Minute Explanation

"SmartBOQ Pro solves three major problems in the Indian construction industry: slow manual estimation, expensive site inspections, and error-prone BOQ preparation. I built a full-stack web application where site engineers upload photos, and our AI instantly detects whether cracks are present, rates the severity as Low/Moderate/High/Critical, and gives an estimated repair cost in INR. The AI uses an ensemble of three pre-trained CNNs achieving 99.58% validation accuracy on 40,000 images. The system also has a complete estimation engine, BOQ generator with PDF/Excel export, AutoCAD DXF parser, and analytics dashboard."

3-Minute Explanation

"Let me walk you through SmartBOQ Pro. The problem I solved: construction companies waste enormous time and money on manual estimation and physical inspections. My solution has two main components. First, the AI Inspection module — I trained four deep learning models using transfer learning from ImageNet. The concrete crack detector is an ensemble of ResNet50, VGG16, and InceptionV3 that achieves 99.58% validation accuracy on 40,000 images. The road damage detector uses MobileNetV2 with fine-tuning. Each inspection gives a severity score, repair urgency, and estimated repair cost in INR. Second, the Estimation Engine — users input building dimensions, select work types, and the system auto-calculates material quantities, applies 2024 India market rates from our database of 46 materials, and generates a complete Bill of Quantities. The BOQ can be exported as PDF or Excel. Technically, the backend is FastAPI with SQLAlchemy ORM and SQLite. Frontend is React 18 with TypeScript. Authentication uses JWT tokens. The entire system is containerized with Docker for easy deployment."

Key Points to Always Mention

- 99.58% validation accuracy — exact number shows credibility
- 40,000 image dataset — shows scale of work
- Ensemble of 3 models — shows knowledge of advanced techniques
- Transfer Learning from ImageNet — shows understanding of the technique
- Full-stack: FastAPI + React + TensorFlow — shows end-to-end capability
- Production-ready: JWT auth, Docker, PDF export — shows professional quality
- India-specific: INR rates, local construction context — shows domain knowledge

Common Follow-up Questions & How to Answer

Q: Why 3 models in ensemble?

A: Each model has different strengths. ResNet50 handles deep features. VGG16 is good at fine textures. InceptionV3 captures multi-scale patterns. Together they are more robust than any single model.

Q: Why 99.58% and not 100%?

A: 100% would indicate overfitting. 99.58% on unseen validation data shows the model genuinely learned to detect cracks, not memorize training images.

Q: What would you improve?

A: Add GPU support for faster inference. Train EfficientNetB7 for building safety. Add object detection (bounding boxes around cracks). Deploy on cloud with auto-scaling.

Q: How does it handle bad images?

A: Extension whitelist (jpg/png only), 50MB size limit, PIL convert(RGB) handles grayscale/RGBA images. Model returns low confidence for unclear images.

Q: Can it work on mobile?

A: Yes. The React frontend is responsive. For mobile app, the FastAPI backend can serve a Flutter/React Native app with the same REST API.